BORA: Bottleneck-Ordered Runtime Adaptation for Hybrid Quantum-Cloud VQA Execution

Jihoon Kim
Cloud Computing Club, SW Development Department
Gwangju Software Meister High School
Gwangju, Republic of Korea
kimjihoon3106@gmail.com

***Abstract*—Hybrid variational quantum algorithms (VQAs) executed over cloud-accessed backends repeatedly couple a classical optimizer with remote execution services. In this setting, practical wall-clock behavior is shaped not only by algorithm-internal work, but also by execution-path factors such as network round-trip time (RTT), backend load, backend heterogeneity, request granularity, and the observability available for bottleneck attribution. This paper presents BORA, a lightweight supervisory runtime that performs execution-path adaptation in bottleneck order. BORA first routes iterations to a more favorable backend, then activates request-level batching only when RTT amortization is likely to be beneficial, and finally escalates observability only when lightweight telemetry is insufficient for reliable attribution. Across a controlled mock-backend study, we show that execution-path bottlenecks materially affect hybrid VQA runtime, that adaptive routing is the dominant source of end-to-end gain, and that batching is useful primarily as a conditional second-stage control. In the beneficial regime, BORA reduces mean per-iteration latency by up to 35.3% relative to a static baseline while preserving optimization outcome. A mechanism-validation study further shows that BORA's strongest support lies in routing necessity, null-regime restraint, and the supporting role of observability, rather than universal superiority over all adaptive variants. A dual-loop isolation experiment shows exact per-iteration loss-trajectory preservation in the deterministic prototype setting and microsecond-scale control overhead, while a small AWS Braket proof of concept serves as a realism anchor.**

***Index Terms*—hybrid quantum-cloud computing, variational quantum algorithms, runtime adaptation, backend scheduling, batching, observability**

I. Introduction

Variational quantum algorithms (VQAs) are increasingly executed through cloud- and service-oriented infrastructures rather than through direct access to a single local device. This model improves accessibility to scarce quantum resources, but also exposes the user directly to system-level variability such as network round-trip delay, queueing, service-layer contention, and performance heterogeneity across backends [1], [2], [4], [9], [14]. As a result, the wall-clock behavior of the same logical workload can vary substantially depending on the execution path through which it is carried out.

This issue is especially important for VQAs because they are hybrid workloads in which a classical optimization loop repeatedly invokes remote quantum evaluations. The time required for each iteration therefore depends not only on circuit depth and shot count, but also on backend choice, request granularity, and runtime conditions [5]–[7], [9], [12]. Prior work has addressed parts of this broader problem through measurement reduction, state-preparation minimization, partial compilation, or cloud-level scheduling [2]–[7], [10], [11]. However, these approaches do not directly address a narrower systems question: how should the repeated execution path of a hybrid VQA workload be controlled at runtime using small, interpretable policies?

This question is practical rather than merely conceptual. In a repeated hybrid loop, the optimizer may generate the same logical kind of work at each iteration, yet the surrounding service path can change from one iteration to the next. Under such conditions, the user experiences runtime not as an abstract complexity bound, but as the cumulative consequence of execution-path decisions. If backend selection, request granularity, and attribution depth are all left fixed, then the runtime may repeatedly pay avoidable transport or queueing costs. Conversely, if every control is made aggressively adaptive without structure, the runtime may become noisy, overreactive, or difficult to interpret. A useful supervisory mechanism should therefore be lightweight, ordered, and conservative.

Seen this way, the problem addressed in this paper sits between two better-studied layers. It is not purely an algorithmic question about reducing measurement count or changing ansatz structure, and it is not purely a provider-side question about global cloud scheduling. Instead, it concerns a narrower execution layer that is visible to the application-facing runtime: which backend to use among a bounded candidate set, when to aggregate submissions, and how much observability is needed before reacting to an apparent slowdown. This layer is small enough to be implementable with lightweight rules, but large enough to affect end-to-end runtime materially.

To address this gap, we propose BORA (Bottleneck-Ordered Runtime Adaptation), a lightweight runtime mechanism for hybrid quantum-cloud VQA execution. BORA structures adaptation in bottleneck order: it first selects the more favorable backend for the current iteration, then applies request-level batching only when RTT amortization is expected to be beneficial, and escalates observability only when lightweight telemetry is insufficient for reliable attribution. BORA is not a new optimizer; it is a supervisory runtime layer that improves execution efficiency without changing the semantic role of the optimization loop.

The paper is organized around four practical questions. First, how sensitive is hybrid VQA iteration time to RTT, backend load, and execution-path choice? Second, can lightweight adaptive routing and conditional batching provide meaningful end-to-end gain over static baselines? Third, when is richer observability actually necessary? Fourth, can such runtime adaptation operate with sufficiently small control cost while preserving the optimization trajectory in the evaluated deterministic setting? These questions motivate a staged experimental program rather than a single headline result.

The main contributions of this paper are as follows. First, we show that execution-path bottlenecks such as RTT, backend load, and attribution ambiguity materially affect hybrid VQA wall-clock behavior. Second, we propose BORA, a small structured runtime mechanism that combines backend routing, conditional batching, and selective observability escalation. Third, we show through integrated evaluation, ablation, and dual-loop isolation that BORA delivers meaningful end-to-end gain while preserving optimization behavior in the deterministic prototype setting. Fourth, we support the practical relevance of the problem setting through a small real-provider realism anchor on AWS Braket. More broadly, the paper argues that this narrow runtime-decision layer is itself a worthwhile systems target under conservative claims.

II. BORA Design and Experimental Setup

Hybrid quantum-cloud VQA execution can be viewed as a distributed iterative procedure in which the optimizer proposes parameters, remote evaluation requests are issued, and the resulting observables are aggregated into the next loss update. In this setting, per-iteration wall-clock behavior depends not only on algorithm-internal work, but also on execution-path terms such as client-side orchestration, network transit, remote queueing, and service-side processing. Conceptually, the resulting latency can be expressed as

$$T_{\text{iter}} \approx T_{\text{client}} + T_{\text{transit}} + T_{\text{queue}} + T_{\text{service}} + T_{\text{return}}. \quad (1)$$

This decomposition is useful because it makes clear that execution inefficiency may arise from different bottlenecks under different conditions, and that not all runtime controls should be treated as equally primary. A route choice affects the dominant backend-side term. A batching choice changes how often transport and submission overhead are paid. An observability choice affects how confidently the runtime can attribute a slowdown before reacting. BORA is therefore built around the principle that these controls should not be activated symmetrically.

BORA is built around an explicit separation between the *solution path* and the *execution path*. The classical optimizer updates the parameter vector Θ according to observed loss values, while the runtime loop determines where and how each iteration is executed. In this sense, BORA should be understood not as a mechanism for finding a better solution, but as one for executing the same search process over a more favorable execution path. This distinction is central to the paper's interpretation of correctness: the runtime is allowed to alter the cost of each iteration, but not the logical role of the iteration itself.

The design follows a simple ordered structure: *route first, batch second, observe selectively*. Routing is primary because the largest execution-time differences often begin with backend choice itself. Batching is secondary because its benefit appears mainly when the selected backend lies in a latency-amortization regime. Observability is supporting because richer telemetry is useful primarily in ambiguity scenarios where lightweight attribution is insufficient. The ordering matters. If a poor backend is selected, batching alone may at best partially amortize a bad choice. If attribution is already clear from lightweight telemetry, escalating observability by default adds complexity without clear decision value. By contrast, route-first adaptation provides a disciplined first response that can expose when batching is worth activating.

Figure 1 illustrates this structure. The optimizer advances the solution path, while BORA acts as a thin supervisory layer over the execution path. For each backend $b \in \mathcal{B}$ in the bounded candidate set, the runtime maintains an estimated latency state updated by an exponentially weighted moving average (EWMA):

$$\hat{L}_b^{(t)} = \alpha L_b^{(t)} + (1-\alpha)\hat{L}_b^{(t-1)} \quad (2)$$

The runtime then selects the backend with the smallest estimated latency:

$$b_t = \arg\min_{b \in \mathcal{B}} \hat{L}_b^{(t)} \quad (3)$$

Batching is controlled by a simple threshold rule:

$$k_t = \begin{cases} k_{\max}, & \text{if } \mathrm{RTT}(b_t) \geq \tau_{\text{rtt}}, \\ 1, & \text{otherwise.} \end{cases} \quad (4)$$

This keeps batching as a structurally simple second-stage control rather than an independent noisy loop. Importantly, $\mathcal{B}$ denotes a bounded candidate set rather than an unconstrained global universe of all possible backends, and batching refers to request-level submission aggregation rather than circuit concatenation or qubit packing.

Algorithm 1 summarizes the per-iteration decision logic.

The prototype is implemented as a minimal artifact-oriented runtime over reproducible mock backends deployed in an EC2-based environment. The mock backend allows controlled injection of RTT, background load, queueing delay, and ambiguity scenarios. This does not attempt to faithfully emulate any one commercial provider. Rather, it provides a controlled environment for causally isolating the effects of routing, batching, and observability. All policy comparisons share the same workload, timing model, and seed sets, so that differences can be attributed to execution-path control rather than workload drift.

The setup is intentionally small. Baseline and batching experiments use nominal, RTT-injected, and load-injected conditions, while heterogeneous-backend experiments use three representative regimes: C0, a null regime in which adaptation should remain restrained; C1, an active regime in which one

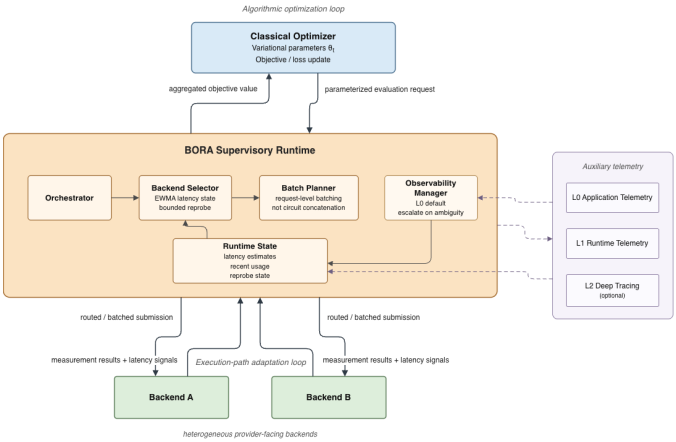

Fig. 1. BORA architecture. The optimizer advances the solution path, while BORA acts as a thin supervisory runtime layer over the execution path through an ordered route–batch–observe pipeline.

Algorithm 1 BORA runtime loop

Input: t, Θ_t, $\mathcal{B}$, runtime state, τ_{rtt}, K, α, $k_{\max}$
Obtain lightweight telemetry from the previous iteration
for each backend $b \in \mathcal{B}$ **do**
  if a new latency observation for b is available **then**
    Update $\hat{L}_b^{(t)} \leftarrow \alpha L_b^{(t)} + (1-\alpha)\hat{L}_b^{(t-1)}$
  end if
end for
if reprobe is due **then**
  Refresh stale backend state
end if
Select $b_t \leftarrow \arg\min_{b\in\mathcal{B}} \hat{L}_b^{(t)}$
if $\mathrm{RTT}(b_t) \geq \tau_{\mathrm{rtt}}$ **then**
  Set $k_t \leftarrow k_{\max}$
else
  Set $k_t \leftarrow 1$
end if
Execute the iteration through backend b_t with batch size k_t
if lightweight telemetry is attribution-ambiguous **then**
  Escalate observability selectively
end if
Update runtime state

backend is clearly inferior; and C2, a transient regime in which backend advantage changes over time. The strongest quantitative evidence in the paper comes from this controlled mock-backend study, while the later AWS Braket experiment is used only as a limited realism anchor rather than as the main mechanism evidence. This separation is deliberate: the mock environment supports causal mechanism isolation, whereas the Braket result establishes that provider-facing latency variability is not merely an artifact of the prototype.

Methodologically, the experiments are staged so that each later claim inherits from a narrower earlier result rather than replacing it. The baseline characterization establishes that execution-path cost is large enough to matter. The batching experiment then tests whether request aggregation can amortize a specific class of transport-dominant cost. The observability experiment asks when L0 telemetry suffices and when richer telemetry becomes necessary for attribution. The heterogeneous-backend experiment then tests route selection alone. Finally, the integrated and ablation studies ask whether these pieces form a defensible ordered mechanism when combined. This staging is important because it prevents the final integrated claim from depending on an opaque bundle of simultaneously changing controls.

A second methodological point is that the paper is intentionally argument-driven rather than benchmark-maximizing. The goal is not to build the largest possible adaptive runtime or to saturate every degree of freedom in the stack. The goal is to test whether a small ordered control policy is enough to yield a defensible systems contribution. This is why the experiments repeatedly compare simple variants that differ in only one primary capability at a time. It also explains why some null or near-null results are retained rather than hidden: they are part of the mechanism argument.

A third point is scope discipline. The prototype is designed to capture repeated provider-facing invocation cost under controlled timing perturbations, not to reproduce every aspect of a commercial quantum service. In particular, the current environment does not model the full richness of provider-internal scheduling policy, calibration refresh dynamics, hardware noise drift, or large-scale multi-tenant interference. The contribution of the study therefore lies in isolating a runtime-decision layer under controlled conditions, rather than in claiming a full-fidelity emulation of production quantum cloud behavior. For the same reason, BORA should be read as an intentionally thin supervisory policy rather than as a complete orchestration substrate.

III. Evaluation

We organize the evaluation around four questions: whether execution path matters, whether adaptive routing is valuable under backend heterogeneity, whether integrated route-first adaptation provides end-to-end gain, and whether the resulting mechanism remains structurally disciplined and lightweight.

A. Execution-Path Bottlenecks

We first characterize the bottlenecks that motivate BORA. Under a static non-adaptive baseline, iteration wall-clock time is highly sensitive to RTT: nominal execution required roughly 199.9 s, while injected RTT of 50 ms and 200 ms increased total runtime to about 220.0 s and 280.1 s, respectively. Background load produced a different pattern, increasing queue wait, tail latency, and variance more than median transport cost. These results show that execution-path variability is substantial enough to justify runtime-side control.

This first result matters because it separates two often conflated questions. One question is whether VQA iteration cost is dominated by quantum-side logical work. Another is whether the observed wall-clock cost of performing that

work is sensitive to surrounding systems factors. The baseline results support the second statement clearly. In particular, the RTT sweep shows a near-monotonic runtime increase as transport delay is injected, while the load condition shows that queueing-related degradation can appear as broader tail and variance effects rather than as a simple median shift.

We then evaluate batching in isolation. Under RTT-dominant conditions, batching clearly helps: batch size 4 reduces total runtime by about 15.4 s under RTT 50 ms and 60.5 s under RTT 200 ms relative to batch size 1. However, its benefit weakens in nominal or load-dominant regimes. This supports the interpretation of batching as a conditional control rather than an always-on optimization. In other words, request aggregation helps mainly when there exists repeated fixed submission overhead worth amortizing. If the dominant slowdown is elsewhere, aggressive batching alone is not a general cure.

Finally, observability experiments show that lightweight L0 telemetry is often sufficient for clear RTT or queueing scenarios, whereas richer observability becomes materially useful in ambiguity scenarios such as client-side stall. In the reduced-scope attribution study, L0 correctly identifies simple RTT and queueing cases, while L1 or L2 is needed to resolve the stall case reliably. The safe interpretation is therefore not that deep observability is universally necessary, but that a selective observability ladder is justified because some ambiguity scenarios are practically important even if they are not the common case. Together, these preliminary experiments motivate BORA's route–batch–observe structure.

To provide a denser supporting view of RTT sensitivity under the same baseline environment, we also repeated the sweep over six RTT settings from 0 to 200 ms. The resulting wall-clock curve increased monotonically, while measured transit delay tracked the injected RTT closely. This supporting result does not replace the main baseline characterization, but reinforces the claim that RTT remains a first-order execution-path variable under controlled conditions.

B. Adaptive Scheduling Under Backend Heterogeneity

Before evaluating the fully integrated runtime, we isolate the value of adaptive backend routing in a heterogeneous two-backend setting. Here, the main result is that backend crossover is real: a backend that is favorable in one condition can become clearly inferior in another. Under such heterogeneity, a single fixed choice is inherently fragile, whereas a lightweight adaptive policy can track the more favorable path under clear asymmetry.

This result does not imply that adaptive routing is always superior to a static policy that somehow already knows the best backend in advance. Rather, it shows that when backend conditions vary, route selection itself becomes the dominant control decision. This provides the direct empirical basis for BORA's route-first design. It also helps explain why batching alone was near-null in later integrated results: if the runtime remains bound to the wrong backend, local amortization of request overhead cannot recover the larger path-selection loss. In that sense, the heterogeneous-backend experiment does not merely precede the integrated study chronologically; it establishes the mechanism ordering that the integrated study later exploits.

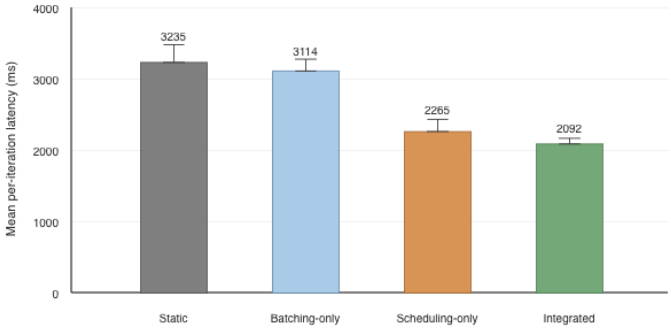

Fig. 2. Policy-level latency comparison in the beneficial C1 regime.

The heterogeneous-backend experiment also helps position the later C2 transient results. In static asymmetry, route adaptation approaches the better path cleanly. In transient asymmetry, however, the useful decision horizon becomes shorter and more sensitive to EWMA responsiveness, stale-state refresh cadence, and the duration of backend advantage windows. This is one reason the paper treats the C2 results as informative but not headline evidence. Put differently, C2 is useful because it shows where the simple runtime begins to strain, not because it supplies the cleanest success case.

C. Integrated Gain and Structural Validation

We next evaluate BORA under integrated runtime conditions. The integrated-runtime experiment compares four variants: `static`, `batching_only`, `scheduling_only`, and `integrated`. The most important result is that the gain structure is routing-dominant and batching-secondary. In the beneficial C1 regime, `integrated` reduces mean per-iteration latency by approximately 35.3% relative to `static`. Most of this gain comes from routing: `scheduling_only` already captures roughly 30.0% of the reduction, while `integrated` contributes an additional 7.6% improvement beyond that. This extra gain should be interpreted not as proof that batching is always powerful in isolation, but as an incremental benefit realized only after routing has first established a favorable backend path.

Figure 2 summarizes this regime-level comparison. The visual pattern is intentionally simple: `static` is clearly worst, `batching_only` remains close to it, `scheduling_only` captures the main improvement, and `integrated` delivers a smaller but consistent additional reduction. This figure is important not because it proves that every control helps equally, but because it makes the gain hierarchy visible at a glance.

Ablation further clarifies the mechanism. The strongest supported result is routing necessity: backend-aware routing is the dominant source of integrated gain. At the same time, BORA's most defensible advantage is not universal superiority over every adaptive variant. In some conditions,

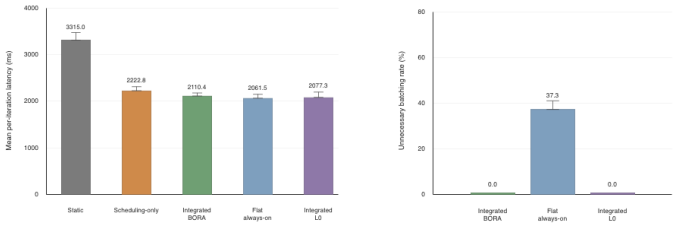

Fig. 3. Experiment 6 ablation results for active-regime latency and null-regime unnecessary batching.

a flat always-on policy is similar to or even slightly faster than `integrated_bora`. However, the null-regime result is more important: in C0, `integrated_bora` suppresses unnecessary batching to essentially 0%, whereas `flat_always_on` incurs unnecessary batching across a substantial range. This means BORA's main strength lies in structural restraint and discipline rather than universal latency dominance.

The observability ablation leads to a similarly scoped interpretation: richer observability is useful primarily as a supporting layer for avoiding policy misattribution, not as an independent source of large mean latency reduction. In this implementation, the measured average observability overhead is near the noise floor. That is useful, but it should not be overstated as proof that richer observability is free.

Experiment 9 complements this picture with a small sensitivity study over $\alpha \in \{0.1, 0.2, 0.5, 0.8\}$ in the transient C2 regime. The results do not imply that $\alpha = 0.2$ is globally optimal, but they do support its interpretation as a conservative balance between responsiveness and stability within the tested range. Lower smoothing reacts more quickly but can become unstable, while heavier smoothing can remain stable but adapt too slowly.

Figure 3 highlights this structural reading. In the active C1 regime, several adaptive variants cluster closely, which cautions against overclaiming universal BORA dominance. In the null C0 regime, however, BORA and the L0-restricted variant remain restrained while the flat always-on variant wastes batching effort. For the paper's overall thesis, this restraint result is at least as important as the absolute best latency bar.

A useful way to read the integrated evidence is therefore as a layered argument. Experiment 5 shows that the full mechanism can produce meaningful end-to-end gain in the right regime. Experiment 6 then asks whether that gain corresponds to the intended mechanism structure rather than to an arbitrary bundle of controls.

D. Correctness, Overhead, and Realism

The dual-loop experiment directly evaluates whether BORA can be interpreted as a supervisory runtime loop that remains separate from the solution path. Across all matched static–integrated pairs, `loss_trajectory_l1 = 0.0` was observed, indicating exact per-iteration loss-trajectory preservation in the deterministic prototype setting. In the active regime, the mean net adaptation overhead after baseline subtraction was only 0.55 μs, while the execution-path benefit exceeded this cost by 4,196$\times$ on average. These results support the interpretation that BORA adds only microsecond-scale control cost while preserving the optimizer's behavior in the evaluated setting.

This experiment strengthens the paper in two ways. First, it makes the solution-path versus execution-path distinction empirically visible rather than merely conceptual. Second, it shows that the runtime loop can remain lightweight in absolute cost terms. The results support microsecond-scale absolute control cost in the tested prototype, but they do not justify claiming a universally negligible overhead fraction under all possible runtimes or denominators.

To complement the controlled mock-backend study, we also include a small real-provider proof of concept on AWS Braket using Rigetti Cepheus-1-108Q. Repeated invocations of the same Bell-state circuit exhibited substantial latency variability, including one large queue spike despite a much smaller median. The observed distribution is informative because it shows that provider-facing repeated execution can display heavy-tail-like behavior even for a tiny workload. This does not prove that the mock environment reproduces vendor internals faithfully. Instead, it supports the narrower realism point that latency variability is not an artifact invented solely by the prototype.

A small sensitivity study over $\alpha \in \{0.1, 0.2, 0.5, 0.8\}$ further suggests that the default $\alpha = 0.2$ should be interpreted not as a globally optimal choice, but as a conservative operating point that balances responsiveness and stability within the tested range. The purpose of this study is to show that the central conclusions do not depend on a single brittle setting, not to argue that BORA has solved parameter tuning in general.

The role separation among these later experiments is worth stating explicitly. Experiment 7 is about correctness isolation and control overhead, not realism. Experiment 8 is about realism anchoring, not mechanism validation. Experiment 9 is about robustness of a single lightweight state-update parameter, not a full tuning campaign.

IV. Discussion, Related Work, and Conclusion

The main implication of this work is that execution path in hybrid quantum-cloud VQA workloads can be treated as an independent systems optimization target. The results show that RTT, backend load, backend heterogeneity, and attribution ambiguity all materially affect wall-clock behavior, and that a small runtime mechanism can mitigate these effects without acting as a new optimizer. In this sense, BORA's value lies less in introducing a large adaptive framework than in showing that a lightweight structured supervisory layer can already be meaningful.

This position connects the paper to three nearby research directions. It is related to quantum-cloud scheduling and resource management because backend heterogeneity and runtime decision-making matter under remote execution [1], [2],

[4]. It is also related to VQA efficiency work on measurement cost and compilation, but differs in that it focuses on deployment-time execution-path control rather than algorithm-internal work reduction or compile-time transformation [5]–[7], [10], [11]. More broadly, it aligns with systems and cross-layer perspectives that treat practical quantum performance as shaped by the interaction between algorithms and the surrounding execution stack [8], [9], [13], [15]–[18].

At the same time, the scope of the present evidence is intentionally limited. The strongest quantitative results come from a controlled mock-backend environment paired with a deterministic expectation model. This improves causal interpretability, but it does not directly include provider-internal scheduling policies, shot-level stochasticity, or broader multi-tenant effects. The small AWS Braket experiment helps position the problem more realistically, but it remains only a realism anchor rather than a replacement for the controlled mechanism study. The paper should therefore be read as making a scoped systems claim about runtime decision structure under controlled heterogeneity, not as claiming full provider-runtime fidelity.

Several additional limitations follow directly from the design. The reprobe rule is fixed and simple rather than confidence-aware. The batching rule is threshold-based rather than queue-aware or congestion-adaptive. The candidate backend set is small and bounded rather than globally open-ended. The present runtime is latency-oriented and single-tenant, and does not explicitly optimize monetary cost, calibration freshness, hardware noise, or multi-agent coordination. None of these omissions invalidate the current study, but they do define its scope clearly.

Some of these limitations are not merely missing features but active design choices that helped keep the mechanism interpretable. Fixed-threshold batching is easier to analyze than a richer congestion controller. Fixed-rate stale-state refresh is easier to reason about than a more aggressive exploration policy. A bounded candidate set avoids conflating backend selection with large-scale discovery or registry management.

These limitations also point to the most natural extensions. One direction is broader backend pools with adaptive reprobe cadence or confidence-aware stale-state refresh. Another is noise-aware or cost-aware routing, where execution quality and financial price become co-equal objectives with latency. A third is compile-time/runtime co-design, in which transport-aware request shaping and algorithm-level workload reduction are considered together rather than separately. A fourth is multi-tenant runtime behavior, where individually rational adaptive agents may interact poorly if they all chase the same temporarily favorable backend.

More generally, the paper argues for a modest systems stance. Hybrid quantum workloads do not need an enormous orchestration framework before runtime structure becomes meaningful. Even a small supervisory layer can matter when it acts on the correct control variables and is evaluated with enough discipline to separate major gains from supporting effects. The ordering principle in BORA is therefore not just an implementation detail; it is the paper's main conceptual claim about how limited runtime complexity should be spent.

Overall, the paper identifies a runtime execution-path decision problem that lies between algorithm-level efficiency and cloud-level scheduling in hybrid quantum systems. BORA provides a first concrete example that this layer can be addressed effectively through an ordered structure of route first, batch second, and observe selectively. The central lesson is not that every adaptive control should always be enabled, but that a small amount of ordered runtime structure is enough to recover meaningful gain while remaining interpretable and scoped.

REFERENCES

[1] G. S. Ravi, Y. Zhan, K. N. Smith, N. Khammassi, J. M. Baker, F. T. Chong, and M. Martonosi, "Quantum Computing in the Cloud: Analyzing Job and Machine Characteristics," in *2021 IEEE Int. Symp. Workload Characterization (IISWC)*, 2021, pp. 39–50.

[2] G. S. Ravi, T. Murali, and F. T. Chong, "Adaptive Job and Resource Management for the Growing Quantum Cloud," in *2021 IEEE Int. Conf. Quantum Computing and Engineering (QCE)*, 2021, pp. 301–312.

[3] S. Chakraborty, Y. Hou, A. Chen, and G. S. Ravi, "Empowering the Quantum Cloud User with QRIO," in *2024 IEEE Int. Symp. Workload Characterization (IISWC)*, 2024.

[4] W. Luo, H. Zhao, C. Zhang, and G. R. Gao, "Adaptive Job Scheduling in Quantum Clouds Using Reinforcement Learning," in *Proc. 54th Int. Conf. Parallel Processing (ICPP)*, 2025, pp. 848–858.

[5] A. Jena, S. Genin, and M. Mosca, "Pauli Partitioning with Respect to Gate Sets for Variational Quantum Eigensolvers," arXiv:1907.07859, 2019.

[6] H.-Y. Huang, R. Kueng, and J. Preskill, "Predicting Many Properties of a Quantum System from Very Few Measurements," *Nature Phys.*, vol. 16, no. 10, pp. 1050–1057, 2020.

[7] S. Herbert and A. Sengupta, "Using Reinforcement Learning to Find Efficient Variational Quantum Circuits," *Phys. Rev. A*, vol. 101, no. 6, p. 062319, 2020.

[8] A. Holmes, M. Johri, G. C. Knee, J. L. Baugh, and M. Mosca, "NISQ++: Boosting Quantum Computing Power by Approximate Quantum Error Correction," arXiv:2004.04794, 2020.

[9] A. Javadi-Abhari, C. G. Almudever, M. Ashraf, et al., "Quantum Computing with Superconducting Qubits: A Systems Perspective," arXiv:2404.17570, 2024.

[10] R. Shaydulin and S. P. Jordan, "Classical Symmetry-Based Subspace Methods for the Variational Quantum Eigensolver," *Quantum Inf. Process.*, vol. 20, no. 12, p. 420, 2021.

[11] S. Majumder, A. A. Fries, S. T. Wang, et al., "Error Mitigation by Circuit Training," *PRX Quantum*, vol. 4, no. 1, p. 010325, 2023.

[12] M. Cerezo, A. Arrasmith, R. Babbush, et al., "Variational Quantum Algorithms," *Nature Rev. Phys.*, vol. 3, pp. 625–644, 2021.

[13] F. T. Chong, D. Franklin, and M. Martonosi, "Programming Languages and Compiler Design for Realistic Quantum Hardware," *Nature*, vol. 549, pp. 180–187, 2017.

[14] A. Y. S. Lam and A. G. Fowler, "Towards Quantum Cloud Computing," in *2018 IEEE Int. Conf. Quantum Computing and Engineering Workshops*, 2018.

[15] S. C. Benjamin, S. Bose, H. Cable, et al., "Brokered Graph-State Quantum Computation," *Proc. Roy. Soc. A*, vol. 461, no. 2063, pp. 3609–3627, 2005.

[16] S. Wehner, D. Elkouss, and R. Hanson, "Quantum Internet: A Vision for the Road Ahead," *Science*, vol. 362, no. 6412, eaam9288, 2018.

[17] C. Elliott, D. Pearson, and G. Troxel, "Quantum Cryptography in Practice," in *Proc. ACM SIGCOMM*, 2003, pp. 227–238.

[18] R. Van Meter and S. J. Devitt, "The Path to Scalable Distributed Quantum Computing," *Computer*, vol. 49, no. 9, pp. 31–42, 2016.